

Deep Reinforcement Learning-Based Task Offloading With Collaborative Inference in UAV-Assisted Mobile Edge Computing Networks

Xiangping Bryce Zhai¹, Member, IEEE, Shuang Fu¹, Changyan Yi¹, Senior Member, IEEE, Zhiquan Liu², Member, IEEE, Chao Dong¹, Senior Member, IEEE, and Chee Wei Tan³, Senior Member, IEEE

Abstract—Intelligent air-ground integration communication is an emerging technology. Uncrewed aerial vehicles (UAVs) serve as mobile edge computing (MEC) servers in large-scale Internet of Things (IoT) applications, alleviating the computational load on ground users. Existing multi-UAV MEC approaches struggle with the complex computation and large data sizes of deep neural network tasks. To address these challenges, we propose a Deep Reinforcement Learning (DRL)-based DNN Partitioning and Dynamic Trajectory Selection (DPDTS) method, which reduces end-to-end latency and system energy consumption through task offloading and collaborative inference. Specifically, we propose an Optimal Partition Point Selection (OPPS) algorithm to minimize transmission overhead by selecting optimal partition points for DNN tasks. Then, we design a fairness-based matching algorithm to optimize user offloading and resource allocation. Finally, OPPS and matching algorithms are integrated to optimize UAV flight trajectories and user transmission power via DRL. The simulation results show that DPDTS outperforms existing benchmark methods in terms of delay and energy efficiency.

Index Terms—Computation offloading, mobile edge computing, multi-UAVs, collaborative inference, deep reinforcement learning.

I. INTRODUCTION

IN RECENT years, the proliferation of mobile smart devices and advances in wireless communication technologies such as 5G and 6G have spawned numerous computing-intensive applications with low latency and low energy consumption requirements, providing users with a better experience and

more features. Mobile edge computing (MEC), an emerging technology that enables mobile ground users (GUs) to execute compute-sensitive applications at the network edge, has attracted a lot of attention [1]. By integrating MEC into the Internet of Things (IoT), computationally intensive and latency-sensitive tasks of IoT devices with limited computational power and energy can be offloaded to the network edge, providing satisfactory Quality of Service (QoS) and Quality of Experience (QoE) for those highly demanding tasks of GUs. Edge intelligence (EI) combines edge computing and artificial intelligence (AI), allowing edge nodes to intelligently process data, optimize resources, and deploy deep learning models for real-time task processing. Recently, the deployment of MEC multi-Uncrewed aerial vehicles (UAVs) has attracted considerable interest due to the inherent flexibility and controllability of UAVs [2]. UAVs equipped with edge servers can continuously provide reliable computation offloading services to users, overcoming geographic constraints and varying deployment environments [3]. Joint multi-dimensional resource management is needed to optimize in the practical implementation for large-scale systems [4].

Deep neural networks (DNNs), as the core technology in machine learning, excel in complex tasks such as image classification, speech recognition, and natural language understanding. However, DNNs typically contain millions of neurons and trillions of connections, which requires substantial computational resources. Their high-resolution raw inputs can also cause significant transmission delays during task offloading [5]. The mobile edge computing framework with multi-user computation offloading and transmission scheduling should be considered for delay-sensitive applications [6]. As a result, most existing approaches cannot scale to multi-UAV MEC for DNN tasks. Tasks are often either fully offloaded, causing excessive transmission latency and energy consumption, processed entirely locally, leading to high computational latency and energy consumption, or arbitrarily partitioned, which is unsuitable for DNN tasks with strong inter-neuron dependencies. In multi-UAV MEC scenarios, the dynamics of UAV positions, limited computing resources, and changing network topologies further affect UAV service quality. Choosing the optimal offloading decision for each GU, efficiently allocating UAV computational resources, and planning flight paths while minimizing system overhead is a typical NP-hard

Received 13 January 2025; revised 3 July 2025 and 23 September 2025; accepted 19 October 2025. Date of publication 13 November 2025; date of current version 26 December 2025. This work was supported in part by the Natural Science Foundation of China under Grant 62531010, in part by the Natural Science Foundation of Jiangsu Province of China under Grant BK20231439 and Grant BK20222012, and in part by the Fundamental Research Funds for the Central Universities under Grant NS2023052. The Associate Editor for this article was S. Wan. (Corresponding author: Shuang Fu.)

Xiangping Bryce Zhai, Shuang Fu, Changyan Yi, and Chao Dong are with Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China, and also with the Key Laboratory of Dynamic Cognitive System of Electromagnetic Spectrum Space, Ministry of Industry and Information Technology, Nanjing 211106, China (e-mail: blueicezhaixp@nuaa.edu.cn; fushuang@nuaa.edu.cn; changyan.yi@nuaa.edu.cn; dch999@gmail.com).

Zhiquan Liu is with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: zqliu@jnu.edu.cn).

Chee Wei Tan is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: cheewei.tan@ntu.edu.sg).

Digital Object Identifier 10.1109/TITS.2025.3629117

1558-0016 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: UNIVERSIDADE ESTADUAL DO CEARA. Downloaded on March 10, 2026 at 21:23:55 UTC from IEEE Xplore. Restrictions apply.

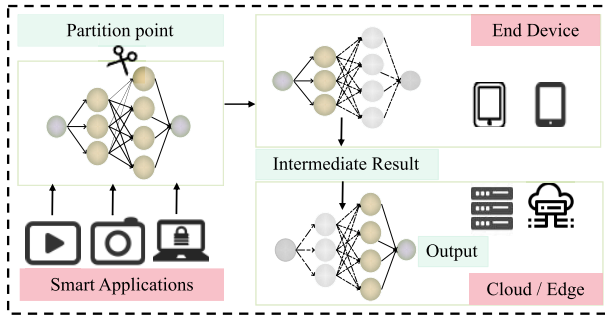


Fig. 1. Collaborative DNN inference in MEC Systems.

problem. Some existing solutions decompose this non-convex optimization problem into multiple convex subproblems and iteratively converge to a suboptimal solution [7], [8], [9].

In a multi-UAV MEC system, DRL enables UAVs not only to leverage powerful computational capabilities but also to make intelligent decisions. Each UAV can plan its next move based on factors such as the GU distribution, current channel conditions, its position, and other environmental factors. Additionally, UAVs' high mobility allows them to quickly and flexibly provide computation offloading services to multiple GUs in a given area. In [10], a deep reinforcement learning based trajectory control algorithm (RAT) based on Deep Deterministic Policy Gradient (DDPG) [11] was proposed to learn the optimal flight strategy of the UAV online and minimize total energy consumption. However, optimization focuses solely on energy consumption, neglecting delay considerations. In [12], an energy efficiency maximization method is introduced using Proximal Policy Optimization (PPO) to efficiently adjust task offloading policies and resource allocation [13]. However, this method applies only to single UAV and sparse GU scenarios, which are not representative of more complex network environments.

In summary, existing multi-UAV MEC systems supporting DNN tasks face main challenges: 1) *reducing transmission latency and processing energy consumption for DNN tasks*, and 2) *rationally allocating UAV computational resources in dynamic environments*. Collaborative inference [14], [15] offers a potential solution to reduce latency and energy consumption by partitioning the DNN into front-end and back-end components, as shown in Fig. 1. This division is based on the fact that the output data in the middle layers of the DNN model are much smaller than the initial inputs, allowing the front-end part to run on mobile devices and the back-end part to run on edge servers.

However, in dynamic multi-UAV MEC scenarios, efficient collaborative DNN inference faces three core challenges: 1) **Conflict between Task Complexity and Resource Constraints**: The strong inter-layer dependencies of DNNs require partition point selection to balance computational/transmission costs, while UAVs' limited resources demand dynamic allocation among users. 2) **Real-Time Adaptability Requirements**: User mobility (e.g., sudden location changes) and fluctuating task arrival patterns require online adaptation of offloading strategies and UAV trajectories. 3) **Complexity of**

Multi-Dimensional Coupled Optimization: Partitioning, resource allocation, and UAV trajectories are highly coupled (e.g., closer UAVs reduce transmission latency but increase energy consumption), making traditional decoupled approaches suboptimal. Existing DRL-based methods [10], [12] partially address these dimensions but fail to resolve their synergistic effects. To further reduce system overhead, we propose a DRL-based DNN Partitioning and Dynamic Trajectory Selection (DPDTS) algorithm. Compared with existing DRL methods, DPDTS pioneers three theoretical breakthroughs: hierarchical collaborative optimization, lightweight real-time decision-making, and environment-aware incremental learning. Our contributions are listed as follows:

- We propose the Optimal Partition Point Selection (OPPS) algorithm, which selects the optimal partition point for DNN models based on task hierarchy features and UAV edge computing capabilities.
- We design a fairness-based matching algorithm to optimize user offloading decisions and allocate UAV computing resources, further reducing delay and energy consumption.
- We present the DPDTS optimization algorithm, which combines OPPS and matching algorithms to adapt to dynamic factors, reduce system costs, and improve UAV service quality.

The remainder of the paper is organized as follows. Section II summarizes some related works. Section III introduces the system model and the formulation of problems. Section IV details the architecture of the method proposed in this paper. In Section V, the simulation results are obtained by numerical experiments. Section VI concludes our work.

II. RELATED WORKS

A. Multi-UAV MEC

In a multi-UAV MEC scenario, the capacity of the UAV's onboard battery and the battery capacity of the ground-user equipment are both limited. Therefore, [16] designed a method based on the Lyapunov optimization technique to analyze the task queue and optimize the offloading decision and the UAV flight path. Reference [17] designed a two-stage online scheduling scheme that dynamically adjusts the CPU frequency and its transmission power of mobile users based on non-cooperative game, in order to minimize the energy consumption of UAVs in the process. In order to provide overall quality of service and user experience when tasks generated by ground equipment have high latency requirements, the authors in [18] investigated the efficient deployment and mobility of UAVs to minimize the maximum delay of all tasks in each time interval. When GU generates intensive tasks with high task volumes and high computational demands, the authors of [19] jointly optimized user associations, UAV paths, and user upload power while satisfying the energy constraints of the UAV and the QoS requirements of each user.

However, solving such optimization problems usually requires lots of computational resources and takes a long time in dynamic environments. The authors in [20] investigated a DRL-based optimization algorithm for joint UAV deployment

and task scheduling to achieve optimal values for the number of UAVs, the hover position of each UAV, and the best strategy for offloading and resource allocation. In [21], a DRL-based dynamic trajectory control algorithm was proposed to learn the optimal flight strategy of the UAV online, and it could also adapt to the dynamic communication conditions between GUs and UAVs to achieve adaptive adjustment of the task offloading and resource allocation. The authors of [22] proposed an evolutionary Stackelberg differential game approach for cloud edge resource allocation, which dynamically optimized computing resource pricing and allocation strategies based on user service selection patterns. This method effectively balances resource utilization and service latency in 5G environments. [23] developed a hybrid DRL-LP framework (PDDQNLP) to jointly optimize 3D trajectory, binary offloading, and time allocation, achieving superior energy efficiency while ensuring fairness among ground users. For scenarios enhanced with RIS, [24] introduced a deep reinforcement learning algorithm that combines the design of the convex hull trajectory and the optimization of the hybrid action space, significantly improving energy efficiency in fixed-wing UAV communications through adaptive trajectory planning and resource scheduling.

B. DNN Partitioning

For DNN models deployed in edge scenarios, traditional approaches usually use techniques such as model pruning [25], binarized neural networks [26], and early exit [27] to reduce the model parameters of the DNN model in order to achieve smaller resources and computational overhead on edge devices. However, the common disadvantage of the above techniques is that they can reduce the accuracy of the model.

To improve real-time partitioning performance in dynamic networks, the authors in [28] proposed a two-stage segmentation strategy to reduce model segmentation complexity. Furthermore, the authors in [29] designed a method that adapts to networks more quickly by using a graph search algorithm based on the neighbor effect. In contrast, the authors of [30] reduced the size of the directed graphs employing model compression, thus reducing the decision time. These studies offer promising solutions for the practical application of collaborative inference in edge scenarios, and a growing body of research explores this field further. In particular, the authors of [31] explored distributed foundation models in 6G networks, integrating pipeline and data parallelism with multi-modal learning to improve collaborative inference efficiency under wireless constraints. The authors in [32] introduced a fine-grained dynamic task scheduling mechanism based on model partitioning techniques, effectively reducing the total energy overhead of the system by applying collaborative inference in edge resource scheduling. For vehicular edge computing, [33] proposed XGBoost-based latency prediction and HLPP-based partitioning algorithms, demonstrating significant improvements in DNN task completion rates and energy efficiency through embedded system experiments.

III. SYSTEM MODEL AND PROBLEM FORMULATION

Consider a multi-UAV assisted MEC system, where the network has K UAVs, each of which is equipped with the edge

computing server and provides communication and computation services for M GUs. Denote $\mathcal{K} = \{1, 2, \dots, K\}$ as the set of UAVs and $\mathcal{M} = \{1, 2, \dots, M\}$ as the set of GUs, and we have $\forall j \in \mathcal{K}, \forall i \in \mathcal{M}$. To keep the position of UAVs relative to users roughly constant, the duration of each round $\mathcal{T}$ is divided into N time slots with a slot length $\Delta = \mathcal{T}/N$, and we have $\forall n \in \{1, 2, \dots, N\}$.

We target DNN-intensive edge computing scenarios, and all tasks generated by ground users require deep neural network inference. Compared with conventional MEC tasks, the computational requirements of such DNN tasks are significantly higher. In time slot n , the i th GU generates some of the same types of DNN tasks $\{w_i, t_i\}$, where w_i represents the task arrival rate and follows the Poisson distribution $w_i \sim \pi(\lambda_i)$, and t_i represents the task type. Our system contains several pre-trained DNN models with different layers and structures. Each type of task has the following attributes, $\{L_{t_i}, C_{l,t_i}, D_{l,t_i}\}, \forall l \in \{0, 1, 2, \dots, L_{t_i}\}$, where l is the index of each layer, L_{t_i} is the number of layers of DNN, C_{l,t_i} is the number of CPU cycles required for the calculation of neurons in each layer, and D_{l,t_i} is the output data size of each layer. When l is 0, it represents the original input. Denote $l_i^*(n)$ as the best partition point for each task of i in the time slot n . Consider a Cartesian coordinate system, in which each GU i is scattered on the ground, and its horizon coordinate is given by $q_i(n) = (x_i(n), y_i(n), 0)$. All UAVs are assumed to fly at a constant height H above the ground. The location of UAV j in the time slot n can be denoted as $p_j(n) = (X_j(n), Y_j(n), H)$. The UAVs circle in the air and cooperatively allocate resources to GUs. Each UAV can communicate with multiple GUs simultaneously during the UAV flight period.

Each UAV serves only GUs within its coverage area and can provide computing services to a maximum of Z users per time slot. The decision variable $\alpha_i(n)$ represents the task offloading of the i th GU. If $\alpha_i(n) = j$, it indicates that the tasks generated in this time slot are offloaded to UAV j . Thus, the following two conditions must be satisfied:

$$\alpha_i(n) = \begin{cases} j, & \text{if } i \text{ offloads the task to } j, \\ 0, & \text{if } i \text{ processes the task locally,} \end{cases} \quad (1)$$

$$\sum_{i=1}^M \mathbb{I}(\alpha_i(n) = j) \leq Z, \quad \forall j. \quad (2)$$

$\mathbb{I}$ is the symbol of the indicator function, which is 1 when the expression following is true and 0 otherwise. In order to effectively reduce task transmission delay and energy consumption, we select the best partition point $l_i^*(n)$ for each task. The first half of each task of the i th GU is executed locally, while the second half of each task of the i th GU is offloaded to the UAV for execution. $l_i^*(n) = 0$ means that the task is completely offloaded to the UAV for execution, while $l_i^*(n) = L_{t_i}$ means executed locally. Each GU can match only one UAV in a time slot, and each UAV selects an appropriate number of user tasks for calculation based on its computing capabilities. We adopt the free space path loss model [34] and set the communication channel between GU and UAV to be mainly a line of sight link, and the communication between GU and UAV follows

the TDMA protocol. Therefore, the ground-to-air channel gain between the i th GU and the j th UAV is expressed as:

$$g_{i,j}(n) = \frac{g_0}{\|q_i(n) - p_j(n)\|^2}, \quad (3)$$

where g_0 represents the channel power gain and $\|q_i(n) - p_j(n)\|$ is the distance between the i th GU and the j th UAV. According to Shannon theory, the uplink data rate is given by:

$$r_{i,j}(n) = \frac{B}{Z} \log_2 \left(1 + \frac{P_i^{off}(n)g_{i,j}(n)}{\sigma^2} \right), \quad (4)$$

where B/Z indicates the fraction of system bandwidth allocated to the user i , and $P_i^{off}(n)$ represents the transmit power from the i th GU and the j th UAV in the time slot n , and σ^2 is the noise power.

Therefore, when the j th UAV handles i th GU's tasks at the n th time slot, we derive the delay for the layers before the partition point of each task to be executed locally:

$$T_{i,j}^{loc}(n) = \frac{\sum_{l=1}^{l_i^*(n)} C_{l,t_i}}{f_i(n)}, \quad (5)$$

where $\sum_{l=1}^{l_i^*(n)} C_{l,t_i}$ is the number of CPU cycles required for local computation and $f_i(n)$ is the computation capability of the i th GU. The intermediate result transmission delay is:

$$T_{i,j}^{trans}(n) = \frac{D_{l_i^*(n),t_i}^*}{r_{i,j}(n)}, \quad (6)$$

where $D_{l_i^*(n),t_i}^*$ is the size of the data that is offloaded to the UAV for processing. Due to the limited computing resources of UAVs, some GUs have to handle their tasks entirely locally. At this time, the result does not need to be transmitted to the UAV, and the size of the result is negligible with $D_{l_i^*(n),t_i}^* = 0$. The delay for the second half of the task to be executed on the UAV is:

$$T_{i,j}^{off}(n) = \frac{\sum_{l=l_i^*(n)+1}^{L_{t_i}} C_{l,t_i}}{b_{i,j}(n)f_j(n)}, \quad (7)$$

where $\sum_{l=l_i^*(n)+1}^{L_{t_i}} C_{l,t_i}$ is the number of CPU cycles required by the UAV to calculate another part of the task, $f_j(n)$ is the computing frequency of the j th UAV, and $b_{i,j}(n)$ is the allocation ratio. The result return time is ignored, so the completion delay of each task is:

$$T_{i,j}^{part}(n) = T_{i,j}^{loc}(n) + T_{i,j}^{trans}(n) + T_{i,j}^{off}(n). \quad (8)$$

Meanwhile, the energy consumption for the layers before the partition point of each task to be executed locally is:

$$E_{i,j}^{loc}(n) = \kappa f_i^3(n) T_{i,j}^{loc}(n), \quad (9)$$

where κ is a constant that represents the effective switched capacitance on the CPU of the GU. The intermediate result transmission energy consumption is:

$$E_{i,j}^{trans}(n) = P_i^{off}(n) T_{i,j}^{trans}(n), \quad (10)$$

The task processing energy consumption is:

$$E_{i,j}^{off}(n) = P_j^{execu}(n) T_{i,j}^{off}(n), \quad (11)$$

where $P_j^{execu}(n)$ is the computing power of UAV, and the flight energy consumption of UAV is:

$$E_{i,j}^{fly}(n) = P_j^{fly}(n) \Delta, \quad (12)$$

where the power of UAVs flight and hovering $P_j^{fly}(n)$ is calculated according to the formula in [35]. So we derive the total energy consumption of each task in the n th time slot:

$$E_{i,j}^{part}(n) = E_{i,j}^{loc}(n) + E_{i,j}^{trans}(n) + E_{i,j}^{off}(n) + E_{i,j}^{fly}(n). \quad (13)$$

Therefore, the weighted sum of latency and energy consumption of each time slot is expressed as follows:

$$e(n) = \sum_{j=1}^K \sum_{i=1}^M \left(\eta E_{i,j}^{part}(n) + (1 - \eta) T_{i,j}^{part}(n) \right) w_i, \quad (14)$$

where η is a weight factor.

We aim to minimize the system's weighted latency and energy consumption, by jointly optimizing the DNN optimal partition point $L^* = \{l_i^*(n), \forall i \in \mathcal{M}, \forall n \in \mathcal{N}\}$, UAV trajectory $\mathcal{U} = \{X_j(n), Y_j(n), V_j(n), \forall j \in \mathcal{K}, \forall n \in \mathcal{N}\}$, computing resource allocation $b = \{b_{i,j}(n), \forall i \in \mathcal{M}, \forall j \in \mathcal{K}, \forall n \in \mathcal{N}\}$, GU transmission power $\mathcal{P} = \{P_i^{off}(n), \forall i \in \mathcal{M}, \forall n \in \mathcal{N}\}$, and offloading decision $\alpha = \{\alpha_i(n), \forall i \in \mathcal{M}, \forall n \in \mathcal{N}\}$. The optimization problem can be formulated as follows:

$$\begin{aligned} \min_{\mathcal{U}, \mathcal{P}, L^*, \alpha, b} \quad & \sum_{n=1}^N e(n) \\ \text{s.t.} \quad & C1 : 0 < \eta < 1, \\ & C2 : \sqrt{(x_i(n) - X_j(n))^2 + (y_i(n) - Y_j(n))^2} \leq d_{max}, \\ & C3 : 0 \leq X_j(n), Y_j(n) \leq D_{max}, \quad \forall j, \forall n, \\ & C4 : 0 \leq \|V_j(n)\| \leq V^{max}, \quad \forall j, \forall n, \\ & C5 : 0 \leq P_i^{off}(n) \leq P_i^{max}, \quad \forall i, \forall n, \\ & C6 : 0 \leq l_i^*(n) \leq L_{t_i}, \quad \forall i, \forall n, \forall t, \\ & C7 : 0 \leq \sum_{n=1}^N \sum_{i=1}^M \left(E_{i,j}^{off}(n) + E_{i,j}^{fly}(n) \right) \leq E_{UAV}^{max}, \\ & \quad \forall j, \\ & C8 : \alpha_i(n) \in \{0, 1, 2, \dots, K\}, \quad \forall i, \forall n, \\ & C9 : 0 \leq \sum_{i=1}^M \mathbb{I}(\alpha_i(n) = j) \leq Z, \quad \forall j, \forall n, \\ & C10 : 0 \leq b_{i,j}(n) \leq 1, \quad \forall i, \forall j, \forall n, \\ & C11 : \sum_{i=1}^M \mathbb{I}(\alpha_i(n) = j) b_{i,j}(n) = 1, \quad \forall j, \forall n. \end{aligned} \quad (15)$$

C2 indicates that the radius of coverage of the UAV signal is d_{max} . The constraints C3 – C4 limit the range and speed of the activity of the UAVs, where D_{max} is the boundary of the activity area, and V^{max} is the maximum flight speed of each UAV. C5 limits each GU's transmit power cannot exceed their respective thresholds P_i^{max} . C6 limits the optimal partition point not to exceed the number of layers of the DNN. C7 limits

the power consumption of each UAV to not exceed the battery capacity E_{UAV}^{max} during service duration. The constraints C8–C9 indicate that the tasks generated by a GU in each time slot are either completely processed locally or partially offloaded to the UAV and each UAV can handle tasks for at most Z GUs. The constraints C10 – C11 limit the proportion of resource allocation.

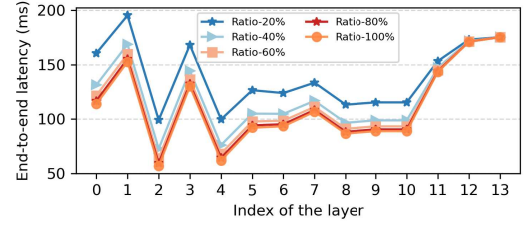
IV. ALGORITHM DESIGN

Due to the non-convexity of (15), we adopt an Alternating Optimization framework and decompose it into three subproblems. The first sub-problem is to find the optimal partition point of each DNN task at every timeslot to reduce the task transmission latency significantly. The second sub-problem is to optimize the offloading strategy and computing resource allocation process, i.e., to find the optimal matching between UAVs and GUs. We propose a matching algorithm based on a fairness factor designed to minimize the energy consumption of the system and the completion time of the task. The third subproblem then is optimizing the trajectory of UAVs and transmitting power of GUs. We formulate UAV trajectory planning and GU transmit power selection as a Markov decision process (MDP), with the negative value of the system optimization objective as a reward. We design a DRL-based algorithm combined with the matching algorithm to solve the problem efficiently and accurately. UAVs act as agents to optimize their strategy and obtain the best action at each time step.

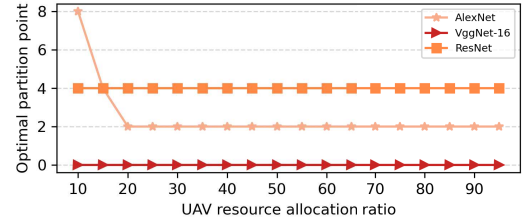
A. Optimal Partition Point Selection Algorithm for DNN Tasks

UAVs can provide computational services to users within their signal coverage to alleviate the computational pressure of users, and different users generate different types of tasks, so the optimal partitioning points are different. Usually, there is a big difference between the computational capacity of UAV servers and user devices, so the selection of DNN task partitioning points is mainly related to the execution latency and energy consumption of user devices, as well as the transmission latency and energy consumption of intermediate data.

When the bandwidth condition is 10 MHz, and the computing frequencies of user devices and UAVs are 1 GHz and 15 GHz, respectively, the relationship between each optional partition point of AlexNet [36] and the end-to-end delay under different resource allocation ratios is demonstrated in Fig. 2(a). The end-to-end delay corresponding to different partition points shows an overall trend under the five resource allocation ratios. Fig. 2(b) demonstrates the selection of optimal partition points under different resource allocation ratios, and the optimal partitions of AlexNet, VGG-16 [37], and ResNet-34 [38] remain unchanged when the ratio is greater than 10%. Therefore, we use $f_j(n)$ to select the optimal partition point. We jointly combine collaborative reasoning and computational offloading and design an Optimal Partition Point Selection (OPPS) algorithm to reduce the system delay and energy consumption. Its pseudo-code is shown in Algorithm 1.



(a) The end-to-end delay versus partition point of AlexNet under different resource allocation ratios.



(b) The optimal partition points of AlexNet, VGG16 and ResNet with different resource allocation ratios.

Fig. 2. The impact of the allocation ratio of UAV computing resources on selecting partition points for several common DNN models.

Algorithm 1 Optimal Partition Point Selection (OPPS)

Input: Task arrival rate w_i , task type t_i , output data size of each layer D_{l,t_i} , and the number of CPU cycles required for the calculation of neurons in each layer C_{l,t_i} , $\forall i \in \mathcal{M}$, $\forall t_i, \forall l \in \{0, 1, 2, \dots, L_{t_i}\}$.

Output: Global partition policy G .

```

1: Initialize the global partition policy  $G$  as empty.
2: for UAV  $j = 1 \rightarrow K$  do
3:   Initialize the set of GUs  $H$  as empty.
4:   Initialize partition policy  $G_{sub}$  as empty.
5:   for GU  $i = 1 \rightarrow M$  do
6:     if C2 and C7 is met then
7:       Add index  $i$  into the set  $H$ .
8:     end if
9:   end for
10:  Initialize  $e^{min}(n)$  as infinite.
11:  for GU  $i \in H$  do
12:    for layer  $l = 0 \rightarrow L_{t_i}$  do
13:      Calculate  $e(n)$  based on (14).
14:      if  $e(n) \leq e^{min}(n)$  then
15:         $e^{min}(n) \leftarrow e(n)$ ,
16:         $l_i^*(n) \leftarrow l$ .
17:      end if
18:    end for
19:    Add the optimal partition  $(i, l_i^*(n))$  into  $G_j$ .
20:  end for
21:  Add  $G_j$  into the global partition policy  $G$ .
22: end for

```

Updated UAV trajectories alter signal coverage, dynamically adjusting the set of serviced users H , which in turn affects partition point selection G_j . The corresponding time complexity is $O(KML^{max})$, where L^{max} is the maximum DNN layers.

Algorithm 2 Matching Algorithm Between UAV and GU

Input: Position and speed of UAVs $\mathcal{U}$, transmission power of GUs $\mathcal{P}$ at time slot n .

Output: Offloading decision α , computing resource allocation strategy b .

```

1: Initialize,  $b = \{b_{i,j}(n) = 1, \forall i, \forall j\}$ .
2: Obtain global partition policy  $G$  by Algorithm 1.
3: for UAV  $j = 1 \rightarrow K$  do
4:   Get the partition policy  $G_{sub}$  from  $G$ .
5:   Initialize the selected GUs queue  $Q_{sub}$ .
6:   for  $(i, l_i^*) \in G_{sub}$  do
7:     Calculate  $e_{i,j}^{dif}(n)$  based on (16).
8:     Add  $(i, e_{i,j}^{dif}(n))$  into  $Q_{sub}$ .
9:   end for
10:  Sort  $Q_{sub}$  by the value  $e_{i,j}^{dif}(n)$  in descending order.
11:  for Element index  $z = 1 \rightarrow Z$  do
12:    Obtain  $(i, e_{i,j}^{dif}(n))$  from  $Q_{sub}$ .
13:     $\alpha_i(n) \leftarrow j$ .
14:    Calculate  $\psi_i(n)$  and  $b_{i,j}(n)$  based on (17) and (18).
15:  end for
16: end for

```

This linear complexity arises from iterating over all possible partition points for each user. The space complexity is $O(M)$, storing only optimal partition points of each user.

B. Matching Algorithm Between UAVs and GUs

In every time slot, GUs within UAVs' signal coverage can choose to offload their tasks to UAV. Nevertheless, since their computational resources are limited, each UAV serves up to only Z GUs. Thus, we must select users for each matching UAV and dynamically allocate resources to obtain a minor system overhead. The matching algorithm between UAVs and GUs is described in Algorithm 2.

Lines 3-9 of Algorithm 2 indicate that for each UAV, it is necessary to first obtain the set of optimal partition points G_j of the tasks of the served users, and then calculate the weighted difference between the energy consumption and delay when the tasks are completely processed locally and offloaded for processing, that is:

$$e_{i,j}^{dif}(n) = w_i \left(\eta E_{i,j}^{dif}(n) + (1 - \eta) T_{i,j}^{dif}(n) \right), \quad (16)$$

where $E_{i,j}^{dif}(n) = E_{i,j}^{loc}(n)|_{l_i^*(n)=L_{t_i}} - E_{i,j}^{part}(n)$ and $T_{i,j}^{dif}(n) = T_{i,j}^{loc}(n)|_{l_i^*(n)=L_{t_i}} - T_{i,j}^{part}(n)$. Then we allocate computation resources for the current UAV, as shown in line 14, based on the CPU frequency required when the task is processed entirely locally, we set a priority-based allocation weight $\psi_i(n)$ indicating the urgency with which the task needs the resources, that is:

$$\psi_i(n) = \frac{\sum_{l=1}^{l_i^*(n)} C_{l,t_i} |_{l_i^*(n)=L_{t_i}}}{\sum_{i'=1}^M \sum_{l=1}^{l_{i'}^*(n)} C_{l,t_{i'}} |_{l_{i'}^*(n)=L_{t_{i'}}}}, \quad \forall i' \in \mathcal{M}, \alpha_i(n) = j, \quad (17)$$

and according to the fairness weight factor $\psi_i(n)$, so we get the resource allocation ratios:

$$b_{i,j}(n) = \frac{\psi_i(n)}{\sum_{i'=1}^M \mathbb{I}(\alpha_{i'}(n) = j) \psi_{i'}(n)}. \quad (18)$$

The time complexity is dominated by sorting operations, i.e., $O(K \cdot M \log M)$, while the space complexity is $O(M \cdot K)$ for priority queues.

C. Trajectory and Transmit Power Optimization Algorithm

After solving the DNN optimal partition point selection problem and the matching problem between UAVs and GUs, we obtain the offloading decision and the UAV server resource allocation ratio for each time slot. Therefore, the simplified problem (15) can be constructed using MDP, consisting of state space $\mathcal{S}$, action space $\mathcal{A}$, state transition probability $\mathcal{P}$, and reward function $\mathcal{R}$. In each time step, the environment is in state $\mathcal{S}_n$, the agent takes action $\mathcal{A}_n$, gets a reward $\mathcal{R}_n$ from the environment, and the environment transitions to state $\mathcal{S}_{n+1}$. In our problem, UAVs act as agents that continuously interact with the dynamic environment to optimize their strategy to maximize the accumulated rewards, i.e., to minimize total energy consumption and reduce task completion latency.

1) State space $\mathcal{S}$: The state of the agent is formulated as:

$$\mathcal{S}_n = \{X_j(n), Y_j(n), w_i, t_i\}, \quad \forall j, \forall i, \quad (19)$$

which represent the current x -coordinate and y -coordinate of each UAV, task arrival rate, and task type of each GU, respectively.

2) Action space $\mathcal{A}$: The action of the agent is defined as:

$$\mathcal{A}_n = \{V_j(n), P_i^{off}(n)\}, \quad \forall j, \forall i, \quad (20)$$

which contains the vector speed of each UAV and the transmission power of GU choosing to offload its task in each time slot.

3) Reward function $\mathcal{R}$: The reward of the agent is defined as:

$$\mathcal{R}_n = -e(n) - \chi(n), \quad (21)$$

where the former part is the negative of the system's optimization goal, and the latter is the penalty for the UAVs approaching or exceeding the activity range.

DPDTS adopts a training mechanism for the twin delayed deep deterministic policy gradient (TD3) [39] with offline replay buffers and delayed updates to address sample correlation and Q -value overestimation problems. The main network includes two critics and an actor, supported by a target network to ensure stability. The actor updates less frequently than the critics, allowing for error minimization before strategy adjustments. Our proposed DPDTS approach is summarized in Algorithm 3 and is shown in Fig. 3. DPDTS combines OPPS and the matching algorithm to optimize UAV trajectories and GU transmission power per time slot. It is divided into four steps: experience collection, offline playback, updating the policy network, and updating the target network. First, through interaction with our mission scenario (the environment), experience samples $(\mathcal{S}_n, \mathcal{A}_n, \mathcal{R}_n, \mathcal{S}_{n+1})$ are collected and stored in

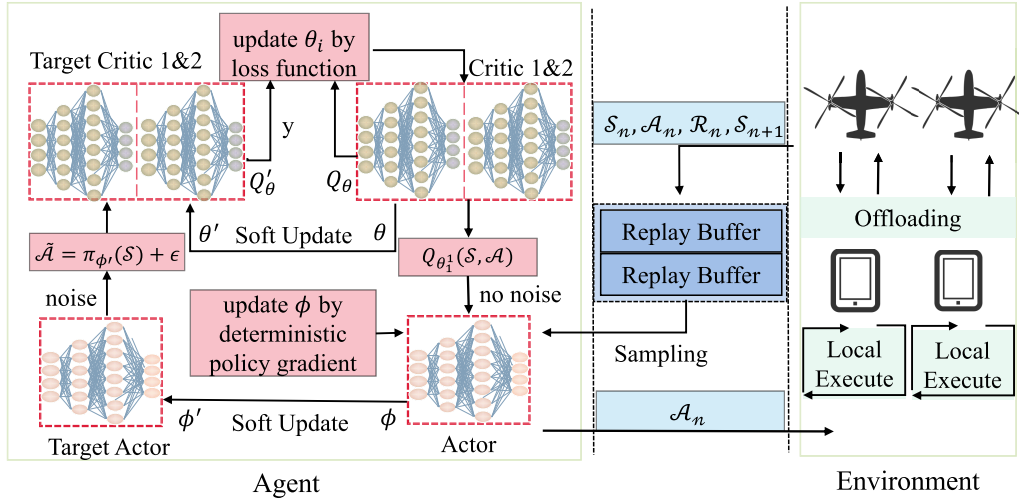


Fig. 3. Architecture of the proposed DPPTS scheme.

Algorithm 3 DNN Partitioning and Dynamic Trajectory Selection (DPPTS)

- 1: Initialize the agent's critic network $Q_{\theta_1}, Q_{\theta_2}$ and actor network π_{ϕ} , and initialize target networks' parameters $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$.
- 2: Initialize replay buffer $\mathcal{B}$, its size L , mini-batch size L_b .
- 3: **for** $episode = 1 \rightarrow num$ **do**
- 4: **for** time slot $n = 1 \rightarrow N$ **do**
- 5: Obtain global partition policy G by Algorithm 1.
- 6: Obtain the offloading decision α by Algorithm 2.
- 7: Obtain state S_n .
- 8: Execute $\mathcal{A}_n$ with exploration noise according to (22).
- 9: Obtain next state S_{n+1} .
- 10: Calculate the reward DELZZ-DEL $\mathcal{R}_n$ based on G, α and UAV's locations.
- 11: Store the transition DELZZ-DEL $(S_n, \mathcal{A}_n, \mathcal{R}_n, S_{n+1})$ into replay buffer $\mathcal{B}$.
- 12: Sampling L_b of L transitions from $\mathcal{B}$.
- 13: Update critic parameters θ_1, θ_2 according to (23) and (24).
- 14: **if** $episode \bmod d$ **then**
- 15: Update parameters of actor network according to (25).
- 16: Update parameters of three target networks according to (26) and (27).
- 17: **end if**
- 18: **end for**
- 19: **end for**

the replay buffer of the experience, as shown in lines 5-11. Then, a mini-batch L_b of experience samples is randomly sampled from the replay buffer and used to update the policy and target networks as shown in line 12. We first calculate $\tilde{\mathcal{A}}_n$ in the state S_{n+1} of the target network:

$$\tilde{\mathcal{A}}_n = \pi_{\phi'}(S_{n+1}) + \epsilon, \epsilon \sim clip(N(0, \tilde{\sigma}), -c, c), \quad (22)$$

where ϵ is a Gaussian-distributed but clipped random exploration noise to smooth out the noise added earlier. Then, the critic networks can update their parameters with the temporal difference (TD) target [11]:

$$y = \mathcal{R}_n + \gamma \min_{i=1,2} Q_{\theta'_i}(S_{n+1}, \tilde{\mathcal{A}}_n), \quad (23)$$

$$\mathcal{L}_{\theta_{i'}} = \frac{1}{L_b} \sum_{m=1}^{L_b} (y_m - Q_{\theta_{i'}}(S_{m,n}, \mathcal{A}_{m,n}))^2, i' = 1, 2, \quad (24)$$

where γ is the discount factor for the future Q -value, generally a number less than 1. Each parameter of the main critic network $\theta_{i'}$ is updated with the cost function $\mathcal{L}_{\theta_{i'}}$ to minimize the error between the evaluation value and the target value, as shown in line 13.

The actor network uses delayed updates. For every d steps, the parameter ϕ in the actor network is updated with a deterministic policy gradient as shown in line 15:

$$\nabla_{\phi} \mathcal{J}(\phi) = \frac{1}{L_b} \sum_{m=1}^{L_b} \nabla_{\mathcal{A}_m} Q_{\theta_1}(S_m, \mathcal{A}_m) \nabla_{\phi} \pi_{\phi}(S_m). \quad (25)$$

Finally, the target network is updated using the soft update method as shown in line 16. Introduce a learning rate τ , make a weighted average of the old target network parameters and the new corresponding network parameters, and then assign them to the target network:

$$\theta'_{i'} \leftarrow \tau \theta_{i'} + (1 - \tau) \theta'_{i'}, i' = 1, 2, \quad (26)$$

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'. \quad (27)$$

During the training phase, the off-line training cost depends on the structure of the Actor-Critic networks and is independent of the problem scale. In the online inference phase, the complexity of single-step decision-making is $O(D_L^2)$, where D_L is the dimension of the hidden layer, and only forward propagation is required for real-time execution.

V. SIMULATION RESULTS

In this section, numerical experiments are carried out to evaluate the performance of our proposed algorithm.

TABLE I
SIMULATION AND TRAINING-RELATED PARAMETERS

Description	Parameter	Value
Serving time	$\mathcal{T}$	900 s
Time slot	Δ	0.6 s
Weight	η	0.5
Channel gain	g_0	1.42×10^{-4}
Power of noise	σ^2	-100 dBm
Chip parameter	κ	10^{-28}
Local computing frequency	f_i	1 GHz
Computing frequency in UAV	f_j	15 GHz
Task arrival rate	λ_i	[0, 5]
Task type	t_i	[0, 2]
Computing Power in UAV	P_j^{execu}	0.1 W
Battery capacity	E_{UAV}^{max}	600 J
Training numbers	num	3×10^6
Number of time slots	N	1500
Discount factor	γ	0.99
Learning rate	τ	5×10^{-3}
Small batch size	L_b	256
Buffer size	L	10^5
Delayed update frequency	d	3
Maximum transmission power	P_i^{max}	0.1 W
Learning rate of Actor	lr_a	3×10^{-4}
Learning rate of Critic	lr_c	3×10^{-4}

A. Simulation Settings

In multi-UAV assisted MEC system, GUs with certain computing capabilities are distributed on the ground in an area of $400m \times 400m$. The flying height H of four UAVs is 50m. The maximum flight speed V^{max} is 2m/s aligns with commercial UAV capabilities and ensures stable connectivity while covering 40% of the operational area per time slot, and their initial position is randomly generated at the active boundary. There are three types of DNN tasks generated by users, ResNet-34, VGG-16, and AlexNet, and the number of CPU cycles C_{l,t_i} and the output data size D_{l,t_i} required for each DNN layer are derived by combining the local device inference latency with computational power. Each UAV can provide computing services for up to $Z = 6$ users during the service duration. Different users have different task types and quantities, but the task types generated by the same user are determined. The actor and critic networks have the same network structure. Both consist of three layers, and the number of neurons in the hidden layer D_L is 128. The primary simulation and the settings of the training-related parameters of DPDTS are shown in Table I.

To validate the performance of the proposed DPDTS method, the following four algorithms are selected as benchmarks for comparison:

- 1) **DTS**: This method is the DPDTS algorithm without considering collaborative inference, where the computation tasks adopt a binary offloading strategy, and the offloading ratio can only be 1 or 0. This method validates the necessity of dynamic partitioning.

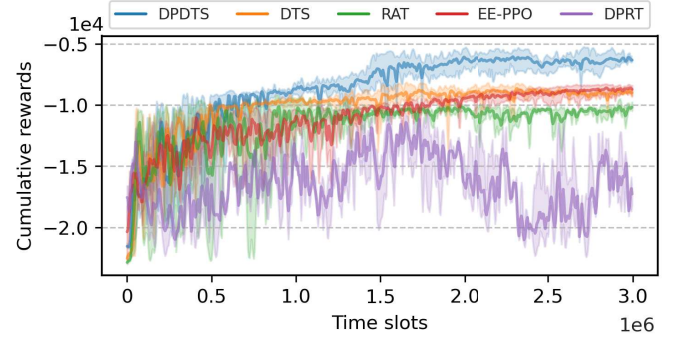


Fig. 4. The cumulative rewards versus time slots.

- 2) **RAT** [10]: This method optimizes task offloading for ground users and DDPG-based UAV trajectories, using a binary offloading strategy for computation tasks and representing classic DRL approaches.
- 3) **EE-PPO** [12]: This method maximizes the average energy efficiency using PPO and enables dynamic adjustment of the task offloading strategy.
- 4) **DPRT**: This method is a variation of the proposed DPDTS algorithm without dynamic trajectory planning, where the UAV trajectory and the ground users' transmission power are randomly generated in each time slot, isolating the contribution of dynamic trajectory planning.

B. Performance Evaluation

First, we evaluate the convergence performance of DPDTS and the other schemes. We set the number of GUs M to be 50, the signal coverage radius d_{max} of each UAV to be 50m, and the communication bandwidth to be 10 MHz. As shown in Fig. 4, as the number of training steps increases, the strategy of the DPDTS agent gradually improves, and the fluctuation in performance has a clear downward trend. Finally, it can obtain a relatively stable reward value, indicating that it has been trained as an agent to make the best real-time decisions. It is easy to see that DPDTS has the highest overall reward and a faster convergence speed. DTS, RAT, and EE-PPO have higher system costs since they do not partition the DNN tasks and have higher task transmission overheads, so the final converged reward is lower, and the performance is not as good as that of DPDTS. Moreover, DPRT has lower performance due to irregular exploration, which makes the reward curve constantly oscillate, making the performance worse.

Then, as shown in Fig. 5, we analyze the total energy consumption of the system as well as the average task processing latency per user obtained for DPDTS and other schemes when the radius of coverage of the UAV varies, and the evaluation data are the average values obtained by the different methods over 50 episodes. The larger the radius of the UAV signal, the more GUs can offload tasks with high complexity to the UAV for execution. Thus, the system energy consumption and the average task latency show a decreasing trend. Fig. 5(a) shows that because DPDTS can efficiently perform collaborative inference and computation offloading of tasks for GUs, it

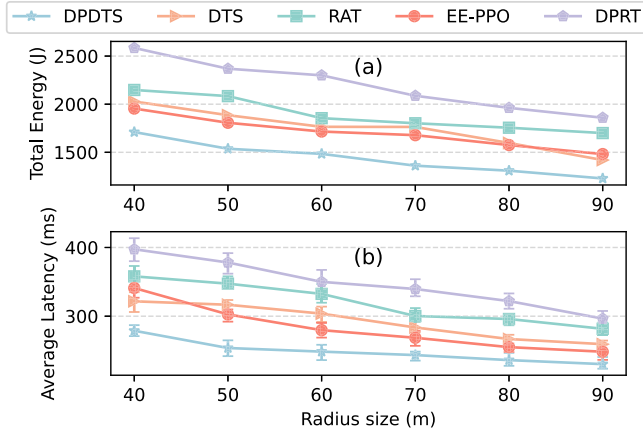


Fig. 5. The total energy consumption and average task latency versus coverage radius size of UAVs.

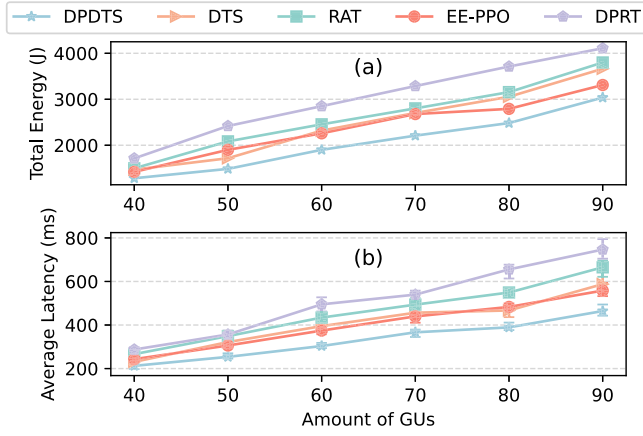


Fig. 6. The total energy consumption and average task latency versus amount of GUs.

produces the lowest system energy consumption. Especially when the radius is 50, DPDTS consumes 1536.9J, which is reduced by 22.6%, 35.5%, 17.6%, and 54.2% compared to DTS, RAT, EE-PPO, and DPRT, respectively. Fig. 5(b) shows that because DPDTS employs DNN partitioning to significantly reduce the transmission delay of the task, the task is processed the fastest with the lowest average time per user. In particular, when the UAV signal coverage radius is 50, the average processing delay per user is 253.7ms, which is 24.8%, 36.8%, 19.26%, and 49.1% faster than the time used by the other schemes, respectively.

As shown in Fig. 6, we further analyze the total system energy consumption and the average task processing latency per user obtained for DPDTS and other schemes when the number of GUs increases. Since UAVs have limited computational resources, more GUs mean that more tasks must be processed locally. Thus, both the system energy consumption and the average task latency tend to increase. Fig. 6(a) shows that because DPDTS can dynamically obtain the partition point that minimizes the end-to-end delay, the energy consumption of the DPDTS system is the lowest, no matter how the number of users changes. Especially when the number of GUs is

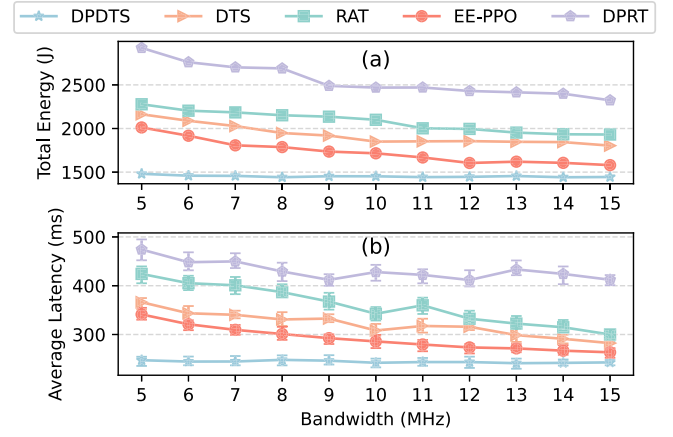


Fig. 7. The total energy consumption and average task latency versus bandwidth.

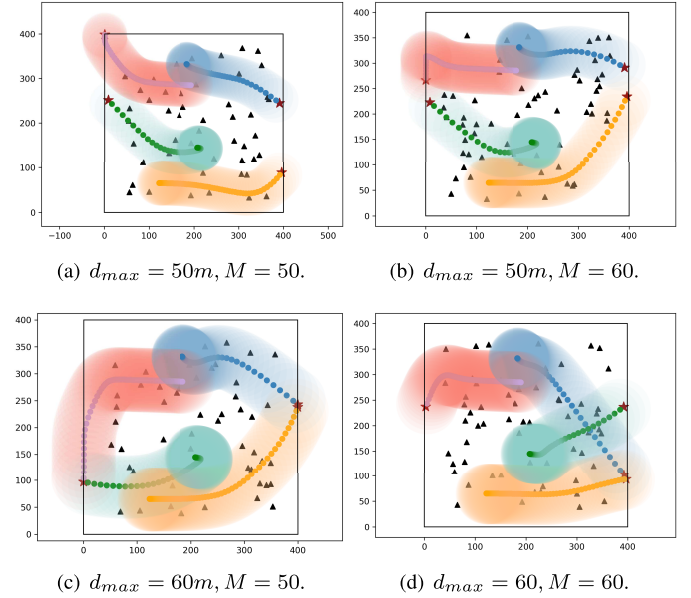


Fig. 8. Four situations of trajectories of four UAVs.

70, the energy consumption generated by DPDTS is 2209.0J, which is 21.3%, 26.7%, 17.8%, and 48.9% lower than that of DTS, RAT, EE-PPO, and DPRT, respectively. Fig. 6(b) shows that DPDTS has the lowest end-to-end delay with varying numbers of GUs, with an average user processing delay of 389ms for a user amount of 80, which is 20.0%, 41.2%, 24.2%, and 68.5% faster than the time used by the other schemes, respectively.

Fig. 7 shows the total system energy consumption and each user's average task processing delay obtained by DPDTS and other schemes under different communication bandwidths. The larger the bandwidth, the higher the data transmission rate, and the transmission delay and transmission energy consumption required for task offloading are reduced. Therefore, both energy consumption and delay show a downward trend. Under various bandwidth conditions, DPDTS performs better. Compared to DTS, RAT, EE-PPO, and DPRT, DPDTS reduces energy consumption by 35.1%, 49.2%, 23.9%, and 66.4% on

average and reduces average delay by 27.5%, 31.7%, 18.2%, and 57.1% on average.

Fig. 8 shows the flight trajectories obtained by DPDTS in four test scenarios, where the four colored rings represent the flight paths of the four UAVs, the stars are randomly generated initial positions at the edges, and the triangles represent the GUs. The radius of coverage of the UAV signal are both 50 m in Figs. 8 (a) and (b), while are 60 m in Figs. 8 (c) and (d). In both Fig. 8(a) and Fig. 8(c), the number of ground users is 50, while there are 60 users in the scenarios of Fig. 8(b) and Fig. 8(d). As shown in Fig. 8, in the duration of service, UAVs departing from the edge tend to fly to the task-intensive area and then explore the best location so that more users can offload tasks and the overall overhead of the system is less. At the same time, for the user equipment that is far away and sparsely distributed, considering the global energy consumption and delay, UAV will choose to continue to hover in the place with more GUs to reduce the path loss and ensure the efficiency of computing resource allocation.

VI. CONCLUSION

In the research on multi-UAV-assisted task offloading for ground users, existing solutions often exhibit poor performance when extended to scenarios of Deep Neural Network (DNN) tasks with high computational complexity and high transmission latency. Therefore, the trajectory optimization algorithm named DPDTS is proposed by combining collaborative inference and task offloading based on user scheduling and task segmentation point selection. The aim is to achieve rational resource allocation and minimize system energy consumption and total latency. DPDTS is a reinforcement learning decision-making method based on the optimal segmentation points of DNNs and fairness factors. It can continuously increase rewards and effectively reduce the latency and energy consumption of DNN tasks in the MEC system. Simulation experiments are conducted to verify the superiority of the proposed scheme in terms of performance and complexity, and it is compared with some existing benchmark schemes based on reinforcement learning. The simulation results show that the proposed method performs better. However, there are still some issues in the current research, such as the insufficient scalability of the algorithm and the gap between the simulation experiments and the actual scenarios. In response to these problems, future research needs to conduct a comprehensive and in-depth analysis and actively explore potential solutions on this basis.

REFERENCES

- [1] A. Telikani, A. Sarkar, B. Du, and J. Shen, "Machine learning for UAV-aided ITS: A review with comparative study," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 15388–15406, Nov. 2024.
- [2] T. Gong, L. Zhu, F. R. Yu, and T. Tang, "Edge intelligence in intelligent transportation systems: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 8919–8944, Sep. 2023.
- [3] G. Li and J. Cai, "An online incentive mechanism for collaborative task offloading in mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 624–636, Jan. 2020.
- [4] C. Yi, S. Huang, and J. Cai, "Joint resource allocation for device-to-device communication assisted fog computing," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 1076–1091, Mar. 2021.
- [5] A. Haydari and Y. Yilmaz, "Deep reinforcement learning for intelligent transportation systems: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 1, pp. 11–32, Jan. 2022.
- [6] C. Yi, J. Cai, and Z. Su, "A multi-user mobile computation offloading and transmission scheduling mechanism for delay-sensitive applications," *IEEE Trans. Mobile Comput.*, vol. 19, no. 1, pp. 29–43, Jan. 2020.
- [7] J. Su, S. Yu, B. Li, and Y. Ye, "Distributed and collective intelligence for computation offloading in aerial edge networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7516–7526, Jul. 2023.
- [8] L. Lyu, F. Zeng, Z. Xiao, C. Zhang, H. Jiang, and V. Havyarimana, "Computation bits maximization in UAV-enabled mobile-edge computing system," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 10640–10651, Jul. 2022.
- [9] G. Zheng, C. Xu, M. Wen, and X. Zhao, "Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10934–10947, Oct. 2022.
- [10] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.
- [11] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [12] B. Li, W. Liu, W. Xie, and X. Li, "Energy-efficient task offloading and trajectory planning in UAV-enabled mobile edge computing networks," *Comput. Netw.*, vol. 234, Oct. 2023, Art. no. 109940.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [14] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Comput. Archit. News*, vol. 45, no. 1, pp. 615–629, 2017.
- [15] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive DNN surgery for inference acceleration on the edge," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1423–1431.
- [16] J. Zhang et al., "Stochastic computation offloading and trajectory scheduling for UAV-assisted mobile edge computing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3688–3699, Apr. 2019.
- [17] W. Lin, T. Huang, X. Li, F. Shi, X. Wang, and C.-H. Hsu, "Energy-efficient computation offloading for UAV-assisted MEC: A two-stage optimization scheme," *ACM Trans. Internet Technol.*, vol. 22, no. 1, pp. 1–23, Feb. 2022.
- [18] Q. Wu, Y. Zeng, and R. Zhang, "Joint trajectory and communication design for multi-UAV enabled wireless networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 2109–2121, Mar. 2018.
- [19] Y. Qian, F. Wang, J. Li, L. Shi, K. Cai, and F. Shu, "User association and path planning for UAV-aided mobile edge computing with energy restriction," *IEEE Wireless Commun. Lett.*, vol. 8, no. 5, pp. 1312–1315, Oct. 2019.
- [20] D. Wei, J. Ma, L. Luo, Y. Wang, L. He, and X. Li, "Computation offloading over multi-UAV MEC network: A distributed deep reinforcement learning approach," *Comput. Netw.*, vol. 199, Nov. 2021, Art. no. 108439.
- [21] Q. Luo, T. H. Luan, W. Shi, and P. Fan, "Deep reinforcement learning based computation offloading and trajectory planning for multi-UAV cooperative target search," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 504–520, Feb. 2023.
- [22] J. Du, C. Jiang, A. Benslimane, S. Guo, and Y. Ren, "SDN-based resource allocation in edge and cloud computing systems: An evolutionary Stackelberg differential game approach," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1613–1628, Aug. 2022.
- [23] N. Lin, H. Tang, L. Zhao, S. Wan, A. Hawbani, and M. Guizani, "A PDDQNLP algorithm for energy efficient computation offloading in UAV-assisted MEC," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8876–8890, Dec. 2023.
- [24] N. Lin et al., "Green communications: RIS-assisted fixed-wing UAV coverage scheme based on deep reinforcement learning," *IEEE Internet Things J.*, vol. 12, no. 4, pp. 4115–4127, Feb. 2025.
- [25] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "JointDNN: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 565–576, Feb. 2021.
- [26] P. Wang, X. He, G. Li, T. Zhao, and J. Cheng, "Sparsity-inducing binarized neural networks," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 7, pp. 12192–12199.
- [27] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, "DeeBERT: Dynamic early exiting for accelerating BERT inference," 2020, *arXiv:2004.12993*.

- [28] S. Zhang et al., "Towards real-time cooperative deep inference over the cloud and edge end devices," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 4, no. 2, pp. 1–24, Jun. 2020.
- [29] H. Wang, B. Guo, J. Liu, S. Liu, Y. Wu, and Z. Yu, "Context-aware adaptive surgery: A fast and effective framework for adaptative model partition," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 5, no. 3, pp. 1–22, Sep. 2021.
- [30] R. Yang, Y. Li, H. He, and W. Zhang, "DNN real-time collaborative inference acceleration with mobile edge computing," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2022, pp. 1–8.
- [31] J. Du, T. Lin, C. Jiang, Q. Yang, C. F. Bader, and Z. Han, "Distributed foundation models for multi-modal learning in 6G wireless networks," *IEEE Wireless Commun.*, vol. 31, no. 3, pp. 20–30, Jun. 2024.
- [32] X. Wang, X. Li, N. Wang, and X. Qin, "Fine-grained cloud edge collaborative dynamic task scheduling based on DNN layer-partitioning," in *Proc. 18th Int. Conf. Mobility, Sens. Netw. (MSN)*, Dec. 2022, pp. 155–162.
- [33] C. Li et al., "DNN inference acceleration based on adaptive task partitioning and offloading in embedded VEC," *ACM Trans. Embedded Comput. Syst.*, vol. 24, no. 4, pp. 1–35, Jul. 2025.
- [34] F. Zhou, Y. Wu, R. Q. Hu, and Y. Qian, "Computation rate maximization in UAV-enabled wireless-powered mobile-edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 9, pp. 1927–1941, Sep. 2018.
- [35] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.
- [36] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: [10.1145/3065386](https://doi.org/10.1145/3065386).
- [37] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [39] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.



Xiangping Bryce Zhai (Member, IEEE) received the B.Eng. degree in computer science and technology from Shandong University in 2006 and the Ph.D. degree in computer science from the City University of Hong Kong in 2013. Previously, he was a Post-Doctoral Fellow with the City University of Hong Kong. He is currently an Associate Professor with the College of Artificial Intelligence, Nanjing University of Aeronautics and Astronautics, China. His research interests include Internet of Flying Things, power control, edge computing, resource optimization, and spatial analytics. He has been actively involved in organizing and chairing sessions and has served as a reviewer for several journals and TPC for several international conferences.



Shuang Fu is currently pursuing the master's degree with the School of Artificial Intelligence, Nanjing University of Aeronautics and Astronautics, China, under the supervision of Asso. Prof. Xiangping Bryce Zhai. Her research interests include Uncrewed aerial vehicles and reinforcement learning for mobile edge computing.

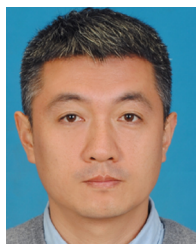


awarded the Changkong Scholar of NUA in 2018 and Chinese Government Award for Outstanding Students Abroad in 2017.

Changyan Yi (Senior Member, IEEE) received the Ph.D. degree from the Department of Electrical and Computer Engineering, University of Manitoba, MB, Canada, in 2018. From 2018 to 2019, he was a Research Associate with the University of Manitoba. He is currently a Professor with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China. His research interests include game theory, queueing theory, machine learning, and their applications in various wireless networks. He was



Zhiqian Liu (Member, IEEE) received the B.S. degree from the School of Science, Xidian University, Xi'an, China, in 2012, and the Ph.D. degree from the School of Computer Science and Technology, Xidian University, in 2017. He is currently an Associate Professor with the College of Cyber Security, Jinan University, Guangzhou, China. His current research interests include trust management, privacy preservation, and artificial intelligence in vehicular networks and UAV networks.



Nanjing University of Aeronautics and Astronautics, Nanjing, China. His current research interests include low-altitude intelligent networks, distributed collaborative intelligence, and electromagnetic large model. He is a member of ACM and IEICE.

Chao Dong (Senior Member, IEEE) received the Ph.D. degree in communication engineering from the PLA University of Science and Technology, Nanjing, China, in 2007. From 2008 to 2011, he was a Post-Doctoral Researcher with the Department of Computer Science and Technology, Nanjing University, China. From 2011 to 2017, he was an Associate Professor with the Institute of Communications Engineering, PLA University of Science and Technology. He is currently a Full Professor with the College of Electronic and Information Engineering,



He served as an Editor for IEEE TRANSACTIONS ON SIGNAL AND INFORMATION PROCESSING, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON COMMUNICATIONS, TPC of IEEE INFOCOM, and ACM SIGMETRICS, and an IEEE ComSoc Distinguished Lecturer.

Chee Wei Tan (Senior Member, IEEE) received the M.A. and Ph.D. degrees in electrical engineering from Princeton University. His research interests include networks, distributed optimization, and generative artificial intelligence (AI). He received the Princeton University Wu Prize for Excellence, the Google Faculty Award, the 2024 IEEE CAI Honorable Mention Award in Foundation Models and Generative AI, several teaching excellence awards, and was selected twice for U.S. National Academy of Engineering China–America Frontiers of Engineering Symposium.